

Modul 03: Service, Contracts, Models und Controller

Ziel: *Durch die Implementierung eines MovieServices (Repository), lernen wir den Umgang mit Dependency Injection und Controllern.*

Tools: *Visual Studio 2022*

Dauer: *~30-45min*

1 Daten-Model anlegen

Erstelle eine Klassenbibliothek mit dem Namen MovieStore. In einem Ordner Models befindet sich folgendes Model und Enum:

```
public class Movie
{
    public long Id { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public decimal Price { get; set; }
    public double IMDBRating { get; set; }
    public DateTime PublishedDate { get; set; }
    public MovieGenre Genre { get; set; }
    public string ImageUrl { get; set; }
}

public enum MovieGenre
{
    Action,
    Adventure,
    Classic,
    Comedy,
    Drama,
    Horror,
    SciFi,
    Thriller
}
```

2 Service anlegen mit CRUD Operationen

Erstelle eine Klasse MovieService. Vorerst enthält er eine Liste mit Beispieleinträgen. Er soll alle CRUD Operationen umfassen.

```
public List<Movie> GetMovies() {}

public Movie GetMovieById(long id) {}

public Movie CreateMovie(Movie movie) {}

public bool UpdateMovie(long id, Movie movie) {}

public bool DeleteMovie(long id) {}
```

3 Contract anlegen

Extrahiere aus dem MovieService ein Interface namens IMovieService, indem Sie den Cursor auf den Klassennamen setzen, Strg + . drücken und aus dem Refactoring Menü den Eintrag extrahieren wählen (Oder Strg + R und Strg + I). Verschiebe Sie das generierte Interface in den Ordner 'Contracts'.

Diese Abstraktion können wir später unsere Abhängigkeit registrieren und sie im Controller verwenden. Ebenso ist sie sehr praktisch um sie in Unit Tests sehr einfach mocken zu können.

4 Einbinden des MovieServices in der Program.cs

Referenzieren Sie die Klassenbibliothek MovieStore als Projektabhängigkeit und binden Sie den MovieService in der Program.cs als Singleton ein:

```
builder.Services.AddSingleton<IMovieService, MovieService>();
builder.Services.AddControllers();
```

5 Movie-Controller erstellen

Legen Sie im Verzeichnis Controller mithilfe des API-Templates den MovieController an und implementieren Sie ihn aus, indem alle CRUD Methoden auf den Service delegieren.

```
[Route("api/[controller]")]
[ApiController]
public class MoviesController : ControllerBase
{
    private readonly IMovieService _service;

    public MoviesController(IMovieService service)
    {
        _service = service;
    }

    [HttpGet]
    public IEnumerable<Movie> GetMovies()
        => _service.GetMovies();

    // usw.
}
```

6 Web API testen

Starten Sie die WebAPI-App und verwenden Swagger, um den Controller mit seinen Methoden zu testen. Alternativ können Sie auch Tools wie Postman, Insomnia, Hoppscotch oder Andere verwenden.

Eine weitere Möglichkeit sind [http-files](#), welche für dieses Lab wie folgt aussehen kann. (Bitte Port noch anpassen)

```
@LabMovieApi_HostAddress = http://localhost:5555
```

```
### Get all movies
```

```
GET {{LabMovieApi_HostAddress}}/api/movies
```

```
Accept: application/json
```

```
### Find movie by id
```

```
GET {{LabMovieApi_HostAddress}}/api/movies/2
```

```
Accept: application/json
```

```
### Create movie
```

```
POST {{LabMovieApi_HostAddress}}/api/movies
```

```
Content-Type: application/json
```

```
Accept: application/json
```

```
{
  "id": 5,
  "title": "Pulp Fiction",
  "description": "Some iconic movie",
  "price": "5.99",
  "genre": 0
}
```

```
### Update movie
```

```
PUT {{LabMovieApi_HostAddress}}/api/movies/2
```

```
Content-Type: application/json
```

```
Accept: application/json
```

```
{
  "id": 2,
  "title": "Rat Race",
  "description": "Very funny movie",
  "price": "5.99",
  "genre": 2
}
```

```
### Delete movie
```

```
Delete {{LabMovieApi_HostAddress}}/api/movies/1
```

```
Accept: application/json
```